Secret References and Controlled Egress for Agent-Written Programs

OpenBot

***Abstract*—An agent can need a password to use a service without needing to read that password. We study a reference-substitution boundary: a trusted host checks and approves a call, reads the secret late, and places it only in the outgoing copy. We define the algorithm, its release events, and a manifest that binds approval to stable identities, argument locations, and a value-version rule. Conditional proofs cover secrecy before release, direct destinations, and approval binding. A counting bound shows why finite text redaction cannot provide secrecy against an allowed hostile tool: an n-bit value can leave through n unchanged boolean results. Executable synthetic tests check substitution and redaction cases; they do not establish all-path product security. We separate the policy revision checked by the host from a stronger revocation guarantee at network send time. The result is a precise account of what references can protect, what an approval must bind, and where a trusted response path is still required.**

***Index Terms*—Secret references, Information flow, Agent programs, Controlled release**

I. Introduction

Agents often need credentials to use a service. Giving a credential to a language model places it in prompts, code, arguments, logs, and answers. A sandbox alone cannot solve this problem: once the program holds the value, any permitted output may copy it.

The baseline defined here takes a narrower approach. It gives the program a string such as ⟦`vault:github/token`⟧. The string names a secret field but contains none of its value. The program can place it in a tool argument. After the program has given up that call, the host checks the target and replaces the reference with the secret. This paper uses **egress** to mean that outward release of a secret from the trusted host to a tool.

The method serves programs that arrange calls without computing on secrets. It does not make a hostile recipient safe. A tool receiving plaintext can retain or disclose it. Result redaction replaces selected secret strings with markers. It reduces common echoes but cannot make a recipient confidential.

We contribute a defined reference baseline, conditional proofs, an approval-binding proposal, and a constructive limit on text filtering. Reproducible synthetic tests check selected properties and attacks. The separate evaluation protocol calls for product boundary tests and performance measurements; those broader studies are not reported here.

The scope is secret-value flow. General permissions, conversational memory, and repeated execution are surrounding assumptions. These algorithms are research definitions, not claims about a deployment.

II. Related work and the design choice

Saltzer and Schroeder describe protected subsystems and the principles of least privilege and complete mediation [1]. They motivate checking each release through one host boundary. Such mediation constrains direct recipients, not their later data use.

Capsicum gives applications operating-system capabilities and a mode with reduced access to global names [2]. Its capabilities are protected tokens of authority. The reference strings defined here are different: a program can write a reference by hand. Their safety comes from host lookup and exit checks, not from making the string impossible to forge. Only the host has the right to read the provider.

Denning models permitted information flows through security classes [3]. Sabelfeld and Myers explain why access checks alone do not control how a program later propagates information [4]. Our exit rule constrains the first release. Text replacement does not track derived values or secret-dependent control branches.

Myers and Liskov give per-owner labels and rules for controlled declassification, which means relaxing a data-flow restriction with suitable authority [5]. An approved tool exit can be viewed as a small, explicit release of this kind. Our defined baseline does not implement their label system. Its policy names connections and tools, and its program values have no runtime secrecy label. A release rule must still define permitted recipients and return data.

Myers, Sabelfeld, and Zdancewic study robust declassification: untrusted code must not steer a release beyond the intended policy [6]. Their work gives a type-based analysis. Our manifest checks one host call; it is not that language-wide result. The agent may propose which item to use, and a trusted decision must give that proposal authority. Checking that the same proposal later runs does not prove that its requested release was wise. This distinction places approval integrity and recipient trust outside the secrecy claim.

Ryoan confines untrusted computation on secret data using a distributed sandbox and hardware enclaves [7]. Its trust problem includes potentially hostile computing platforms. Our trust boundary is different. The method studied here trusts its host and provider, keeps plaintext out of agent code, and sends it to a chosen outside tool. It provides no enclave or remote attestation claim. A sandbox and encrypted storage alone do not imply confidential execution.

CaMeL separates control and data flow and checks tool calls using capabilities [8]. Fides gives an agent-planner model with confidentiality and integrity labels, including ways to hide and inspect data [9]. Both address a broader agent infor-

mation-flow problem. Our scope is the smaller boundary at which an already approved call obtains a credential. We do not claim reference passing or controlled release as new ideas. The contribution is the exact connection between reference syntax, approval identity, value rotation, revocation order, and the remaining response channel. No comparative AgentDojo result is claimed.

III. System and threat model

The model writes a program that runs in an isolated engine. The host holds item metadata, obtains decisions, reads values from a provider, and sends tool calls. A tool receives allowed calls and returns results. Approval comes from a person or policy outside the malicious program. It is not evidence of an honest tool.

An item has a stable identity, an alias, a set of field names, a source, and a set of exits. An exit names a tool connection and either one tool or all its tools. A reference is a string naming an alias and field. Let $S(i, f)$ be the plaintext value of field f of item i. Let $E(i)$ be its exit set. A call contains a connection c, a tool t, and a JSON argument tree a. JSON is a data format with objects, lists, numbers, booleans, and strings.

The model and program may be malicious. They can invent references, copy them, change them, build nested arguments, and adapt future calls to earlier results. A tool may also be malicious, including a tool that has been placed in an allowed exit set. It can return any protocol-valid data and can retain plaintext once received. Tool-provided descriptions and names do not establish trust.

The host, provider, and secret-read path are trusted for the positive claims. Their code, transport protection, configuration, and keys must be sound. The sandbox must keep agent code from host memory and from direct network, file, or process access. The program must not receive the same secret through an unrelated prompt, file, or tool. We exclude a compromised host, provider, operating system, engine escape, and physical memory observation. These exclusions are essential, not hidden claims of protection.

We distinguish a **content observation** from a timing observation. Content includes prompt text, program values, logs returned to the model, tool results, approval views, and public execution records. Provider latency, request sizes, and tool completion times may also carry information. Our positive content claims do not cover those channels. Item names and allowed field names are public metadata within the run, so the method does not hide that a credential exists.

TABLE I
The intended data path. The fourth row is a deliberate secret release.

Boundary	Data allowed across it
Host to program	Aliases, fields, references
Program to approval	Tool and reference arguments
Provider to host	Requested plaintext fields
Host to allowed tool	Arguments with plaintext
Tool to program	Result after finite redaction

IV. Defined baseline

The baseline host maintains an item map and a connection map. Each item has a unique stable identifier, a policy version, declared fields, and allowed exits. Each connection has an identifier and revision. A run begins with a permitted item snapshot. Reading this snapshot exposes names and policy, never field values.

A. Reference grammar and substitution

The host sorts items by stable identifier. It assigns each its requested alias, appending `_2`, `_3`, and so on until the name is unused. Aliases start with a lower-case Latin letter and continue with lower-case letters, digits, or underscores. A reference has the literal form ⟦`vault:alias/field`⟧. The field contains one to 64 Unicode code points and excludes the slash, reference delimiters, and control characters U+0000 through U+001F.

The program receives frozen records of those reference strings. It may concatenate a reference with `Bearer ` or place it in nested JSON values. Encoding the string with base64 encodes the reference text; it does not encode the unknown secret. Reference names need not be unguessable. The host always resolves them against the permitted item snapshot.

Define `scan(a)` as a recursive walk over JSON argument values. For a string, collect every non-overlapping reference matching the grammar, from left to right. For a list, walk its elements in order. For an object, walk its values in stored member order. Ignore object keys, numbers, booleans, and null. Remove duplicate alias-field pairs after preserving their first occurrence. Resolve every pair to a declared field of a permitted item. Reject the entire call if any pair fails. Reject any secret-bearing call to a resource or prompt interface. For a tool call, each referenced item must allow the exact connection and tool, or that connection with a wildcard tool.

Define `sub(a,v)` over the original argument tree and a host-only field-value map. Replace every recognized reference in a string value with its mapped value. Preserve all other text. Walk lists and object values recursively; leave keys unchanged. Use literal replacement, so a dollar sign in a secret has no special meaning. Create a new argument tree and do not scan the replacement text again. Thus a secret that itself resembles a reference does not cause a second read. An unmapped pair causes rejection before sending.

These rules permit an alias to be written by hand, but do not let it bypass lookup. They also make alias stability conditional on the item set. A newly added item can change which object receives a suffix. A later decision should therefore bind stable identities as well as names.

B. Approval, late read, and failure routes

For each call, the host stores its identity, connection and revision, tool, and the original reference arguments. Every call containing a reference requires an explicit decision, including a tool that only reads. The decision view contains those arguments. Approval does not reveal plaintext. The host records a digest of an unambiguous deterministic encoding of the

arguments and target. A later attempt at the same call position must match that digest.

The host wraps each accepted call in a release procedure. It checks exit policy against the permitted item snapshot. Immediately before sending, it also checks the connection revision. It then reads the requested fields, makes a private substituted copy, and invokes the captured tool binding. The internal field reader checks organization and item identity but does not itself check the agent grant or exit. A later read may therefore return current data after the snapshot's policy changed.

A late read chooses a value close to send time. This can be useful for rotation and one-time codes. For a concrete provider example, the official HashiCorp KV version 2 API returns the latest value when a request omits a version [10]. That behavior does not mean approval of a reference is approval of any future replacement item, or that the value cannot rotate again between read and send.

The tool may return a completed JSON result, a typed failure, or a request for more input. Completed results pass through the redactor below. A tool failure carrying JSON also passes through it; if the result no longer fits its required schema, return a fixed public schema error. Client-error text passes through text redaction. Input-schema failures receive fixed public text, because a detailed error could quote the substituted argument. Other error variants pass through unchanged in this defined baseline. A request for more input is refused because its later answer path has no secret context.

C. Finite result redaction

For each secret string, form a list containing the raw value, padded standard base64, unpadded standard base64, unpadded base64url, lower-case hexadecimal, upper-case hexadecimal, percent-encoded URL text, and JSON-escaped string content without its surrounding quotes. Byte encodings use UTF-8. Percent encoding leaves letters, digits, and `-_.!~*'()` unchanged and uses upper-case hexadecimal for other bytes. Remove duplicates, discard forms shorter than four Unicode code points, and order the rest by decreasing length. Break ties by the listed order.

Process secrets in reference-discovery order. For each form, replace every exact occurrence in the current string with `⟦hidden:alias/field⟧`. This is literal text replacement, not decoding or pattern matching. For JSON, visit every string value and object key. Process object members in their original order. If changed keys collide, the last member wins. Numbers, booleans, and null are unchanged. Return and publicly store the filtered tree. A reference-free call has no secret list, so its result is unchanged.

The threshold applies to each form, not once to the original secret. For the illustrative value `ab`, raw `ab` and unpadded `YWI` survive, while padded `YWI=` and hexadecimal `6162` qualify for replacement. A secret equal to a schema key can change the result's structure. A secret equal to part of a hidden marker can even reappear inside that marker. These are derived outcomes.

V. A precise release method

The proposed extension binds review to a complete release request. A string alias alone does not state the full meaning of approval.

At inspection, the host creates a **release manifest**: a record of what will leave and where. It contains the call identity, organization, stable item identities, fields, item-policy versions, connection identity and revision, tool name, and a digest of the complete reference argument tree. It also lists each JSON value location where substitution will occur. The digest is over references and public arguments only. A digest of a short plaintext secret would create a guessing target and is not needed here.

The manifest records one of two value rules. Under a current-value rule, approval allows the value that the same item field supplies when read for this release. The field can rotate again before the transport sends it. Under a fixed-version rule, approval allows only a named provider version. The latter requires provider support and must fail when the version cannot be read. Neither rule silently permits changing the item's source, field meaning, or exit policy after review. Such changes require a new manifest and decision.

A. Encoding and approval identity

The manifest also names the acting principal, the grant revision, and the provider source revision. Every change that affects permission or source meaning must advance a checked revision. Otherwise equal version numbers do not mean equal policy. Call identities are unique within the organization and run. An approval of one call cannot be used for a different call with the same arguments.

The digest needs an injective encoding: two different approved trees must not become the same byte string before hashing. Fix a versioned encoding of typed tree nodes. Encode an object as an ordered list of key and child pairs, an array as an ordered list of children, and a scalar with its type tag. Reject duplicate keys, invalid Unicode, non-finite numbers, and negative zero. Numeric equality means equality of the admitted numeric value, not its original source spelling. Preserve string content and object member order. The location list uses array indexes and object member indexes plus keys in this same tree.

One way to encode these tagged nodes is JSON Canonicalization Scheme, or JCS [11]. JCS sorts object keys and defines string and number encoding. Our object member order remains in arrays of pairs, so that order is not lost. The verified negative-zero erratum informs the rejection rule [12]. Hash the resulting UTF-8 bytes with a fixed protocol tag, `secret-release-v1`, and include the complete manifest. A versioned tag keeps this digest separate from hashes used by another protocol. This is a proposed wire contract, not a claim that ordinary JSON stringification is sufficient.

Immediately before release, the host rechecks the manifest against current metadata and the approved argument tree. It checks that every listed item remains usable, every field remains declared, and every exit remains allowed. It reads only those fields and substitutes only at the listed value locations.

The original argument tree stays intact. The underlying transport must use the checked connection and tool. A connection identifier is not enough if that connection is a general relay to arbitrary user-supplied destinations.

For a tool accepting a URL, path, recipient, or script, review must include those arguments. A narrow adapter may restrict token use to an authentication field and a fixed service origin. This would strengthen the baseline, whose exit rule permits all behavior of the selected tool.

1)	Parse references from the original arguments.
2)	Bind each name to an item, field, and location.
3)	Save and approve the release manifest.
4)	Recheck manifest, policy, and connection.
5)	Read fields under the stated value rule.
6)	Substitute in a private outgoing copy.
7)	Send once through the checked binding.
8)	Filter the answer before public storage.

Fig. 1. Proposed release procedure. "Send once" describes this invocation, not a guarantee about retries or outside effects.

The final stage must cover each return route that can reach the program or a public record. Successes, typed failures, and later interactive answers are separate routes. A route without the release context must be refused or handled by a separate design. Host diagnostics deserve a separate review: calling an error "internal" does not make a log safe for a broader reader. A provider adapter should avoid retaining malformed response bodies in decode failures. Even a short provider message can contain a secret, so safe error handling needs an explicit contract. The provider must remain trusted under the stated model.

B. Release events and revocation time

A trace records `inspect`, `approve`, `read`, `check`, `send`, and `return` events. Each event names the call identity. A `check` event records the manifest and policy revisions it read. A `send` event records the captured connection, tool, and substituted tree in a private audit channel. The public trace omits plaintext and provider bodies. Only `send` releases a field to an outside recipient in this model.

Define the **checked-policy property**: each `send` has an earlier successful `check` for the same manifest, tree, and captured binding. No host mutation may change those call values between the two events. Define the stronger **send-time policy property**: the release also satisfies the policy in force at the send event. The second does not follow from the first. An operator can revoke an exit after the final check and before the transport accepts bytes.

The stronger property needs an ordering rule. For example, the host can hold a policy lease from final check through transfer to a trusted transport. Revocation takes effect after earlier leases close, and new leases read the new revision. The transfer is then the linearization point: the single event used to order a release against revocation. A local database transaction alone cannot make remote receipt atomic with a local policy write. Without a lease or an equivalent protocol, the sound claim names the checked revision and leaves the check-to-send interval outside immediate revocation. This paper does not supply or evaluate such a lease service.

Approval binding is also separate from at-most-once sending. Two concurrent workers may both possess an unchanged approved manifest. To admit at most one host invocation, atomically consume the call identity before send and never resend an uncertain invocation. That rule can lose a call after a crash. Eventual completion without duplicate outside effects needs recipient idempotency or a stronger protocol. The manifest alone gives neither property.

VI. Conditional security claims

A. Secrecy before release

Claim 1. Consider two executions with the same public metadata, code, initial arguments, and scheduling inputs, but different secret values. Before the first value read or secret-dependent outside response, the program's content observations are equal, if the host supplies only references and there is no other source of the secret.

Proof argument. The worker's initial state is built from public names and fixed reference syntax. Changing a secret value changes none of these inputs. Each local program step reads the same state and therefore produces the same next state when random and clock inputs are coupled. Constructing, slicing, hashing, or encoding a reference changes only public text. Host checks before reading use metadata and arguments, which are equal by assumption. Induction over these steps gives equal observations. This proof ends at the first secret-dependent operation. It says nothing about a tool's later answer, and it does not rule out lucky guesses of the value.

B. Confinement of direct release

Claim 2. Every plaintext substitution performed by the checked wrapper is sent only to an exit allowed by its checked metadata, provided every send uses that wrapper, its resolved binding cannot change, and the performed arguments match the checked arguments.

Proof argument. Every substituted field must come from the resolved reference set. Each member passes the field and exit checks before release. The wrapper obtains values only for that set and passes the substituted copy to the captured binding. With no alternate read or send path, there is no other direct release transition. Therefore every such transition has an allowed connection and tool. This is a snapshot claim for the defined baseline. It becomes a current-policy claim only with a sound send-time recheck. It never restricts a recipient's onward sends.

The argument-matching premise matters. A wrapper checked for one request is not a general-purpose secret substitution service. Reusing it with new argument locations could preserve the same fields while changing the meaning of the approved operation. The proposed manifest makes this premise explicit. The abstract protocol requires the recorded argument tree at execution. A concrete implementation must prove that its callers preserve this requirement.

C. Binding a decision to a release

Claim 3. Fix the injective encoding above. Under collision-resistant hashing, an immutable approved manifest, complete relevant revision checks, and complete mediation, a release cannot change its approved reference arguments, item-field bindings, acting principal, or target without rejection, except with the attacker's probability of finding a hash collision. This is a checked-policy claim unless release and revocation also have the ordering rule above.

Proof argument. A changed admitted tree or manifest has different encoded bytes. If its checked digest equals the approved digest, those bytes form a hash collision. Otherwise comparison rejects. Direct revision comparisons reject policy, source, or connection changes before release. Under the current-value rule, the plaintext may rotate without violating the claim because that choice is stated in the manifest. Under the fixed-version rule, the reader must obtain the named version. No numerical collision bound follows from digest length alone; it also needs an explicit hash and attacker model. This result does not establish human understanding of an approval or defeat a misleading but fully visible argument.

D. Limited redaction coverage

Claim 4. Exact replacement removes all occurrences of a listed form in the visited strings if replacements do not contain or create any listed form, and traversal visits every relevant string. This includes keys only when the traversal handles them.

Proof argument. A replacement pass splits on all occurrences of its chosen form and joins with a marker. The chosen form is absent afterward under the stated marker condition. Later passes preserve that absence only under the no-creation condition. Induction over the form list and the JSON tree gives the claim. These conditions are deliberately stronger than an unconditional statement about the defined redactor. A secret can equal part of a marker, and key changes can affect result structure. No proof of general noninterference follows from finite replacement.

E. A response-channel bound and a matching attack

Claim 5. Let a secret be uniform over N possible values. Fix public inputs and program random choices. Suppose the only secret-dependent observation is a sequence of exactly q responses, each chosen from at most m public outcomes. Then any reconstruction procedure has success probability at most $\min\left(1, \frac{m^q}{N}\right)$. Certain recovery of every value requires $m^q \geq N$.

Proof. There are at most m^q response transcripts. Adaptive requests do not add transcripts: with fixed program random choices, the previous responses determine the next request. For each transcript a reconstruction procedure returns at most one correct value. Thus it can be correct for at most m^q of the N equally likely values. A random reconstruction procedure is an average of such fixed procedures and has the same bound. This counting argument excludes time, response count, size, errors, and outside storage as additional channels; if any is observable it must be included in the outcome set.

Corollary. For every n-bit secret, an allowed hostile tool can disclose the full value in n boolean outcomes that the defined redactor leaves unchanged. On query j, return bit j as `true` or `false`. The program reconstructs the bits in order. There are 2^n possible transcripts, which meets the bound with $m = 2$ and $q = n$. One result containing an array of n booleans carries the same information in one tool call; a call-count limit is therefore not a bit limit.

This result holds for any expansion of the list of text encodings while booleans remain unchanged. It is not a new general information-flow theorem. It gives a precise impossibility boundary for this design: direct-release checks and successful whole-token filtering cannot imply secrecy after a hostile recipient receives plaintext. A trusted adapter that maps every response to the same public receipt has one content outcome, but this is useful only when it also closes errors, status, timing, and later-read channels. Most useful tools intentionally reveal some facts, so their permitted response functions must be specified rather than described as fully secret.

VII. Worked traces and attacks

Consider an illustrative item `github/token` with value `ABCD1234`. It may go only to connection `c1`, tool `create_issue`. The program builds an authorization argument from `Bearer ` and its reference. It cannot read `ABCD1234` from that string. The approval record contains the reference and the requested issue title. After approval, the host reads the value and sends `Bearer ABCD1234`. A normal tool returns an issue number. An echo tool returns the authorization string, which the host changes to a hidden marker before it reaches the program.

Now replace that tool with a malicious one. It returns two fields: `left` is `ABCD`, and `right` is `1234`. Neither string contains the full raw secret or one of its listed whole-value encodings. Redaction leaves both fields. The program joins them and obtains all eight characters. Four plus four returned characters reconstruct the eight-character value. No sandbox escape or exit violation is needed.

An even smaller channel returns whether the first secret character belongs to a chosen set. The answer is a boolean, so text replacement cannot remove the leaked fact. Repeated queries can narrow the value if the tool allows them. A digest is also information: it may reveal a short or predictable value by offline guessing even when the full plaintext never appears in the result. The relevant test is what an observer can learn, not whether a substring search finds the token.

TABLE II
ATTACK OUTCOMES UNDER THE STATED HOST ASSUMPTIONS. THESE ARE ANALYTICAL CASES, NOT MEASURED TRIAL RESULTS.

Attempt	Boundary and result
Invent unknown alias	Host lookup rejects the release.
Encode a reference	Only public reference text changes.
Use denied tool	Exit check rejects substitution.
Return full known form	Finite replacement can remove it.
Return pieces or a bit	Information can reach the program.
Write, then read later	Later call may lack secret context.
Change policy while waiting	Needs current checks, not only approval.

A storage tool gives another failure trace. It receives an approved secret-bearing write and returns a harmless receipt. The host has nothing to redact. A later call with no reference reads the stored value. A reference-free call has no per-call secret list for redaction. Per-call filtering cannot enforce a lifetime restriction on an outside store.

Short values create a direct tradeoff. Replacing every occurrence of a one-character secret would destroy ordinary text and reveal which character was selected through the redaction pattern. Keeping short forms preserves output but leaves a channel. A numeric result can also carry secret digits as numbers, which the string traversal does not change. Unicode changes, new encodings, encryption under an attacker key, and text in media add further uncovered forms. Expanding the list addresses selected cases but does not close the class.

Policy races are a separate failure class. Suppose a decision waits while an operator removes an exit or changes an item's source. A prior check can still hold old metadata while a later read returns a current value. The baseline checks the saved snapshot but reads current data, so it does not establish current-policy enforcement across that interval. A version-bound manifest and immediate recheck would detect the relevant change, subject to the ordinary race between check and send. A stronger atomic lease would need a defined policy and transport protocol; we do not claim one exists.

VIII. Evaluation protocol

The proposed evaluation asks whether plaintext stays out of the program before release, whether releases match exits and approvals, and what information survives filtering. Report these questions separately; rejection and recipient trust are different properties.

Use an isolated test provider and tool server with generated credentials that grant no real access. The provider records read requests. The tool records exact received bytes in a restricted test log. Instrument worker messages, approval payloads, operation records, model prompts, public errors, and returned logs. Keep private tool logs separate from program-visible data to avoid a test-created leak.

The corpus should vary secret lengths below, at, and above the four-character threshold. Include high-entropy tokens, dictionary words, decimal codes, non-ASCII text, quotes, delimiter-like strings, shared prefixes, and values equal to schema keys or marker words. Vary reference location, repetition, multiple items, alias clashes, allowed and denied tools, wildcard exits, and provider failures. Generate nested lists and objects, while keeping object-key references as a distinct case because substitution does not handle them.

Compare four implementations. The first gives raw secrets to the program and uses no filtering. The second gives raw secrets but filters results. The third uses references and exit checks without result filtering. The fourth is the defined reference baseline with filtering. A fifth variant adds the proposed manifest and is reported separately. All variants use the same sandbox, provider, approval inputs, and tool behavior.

For release checks, count provider reads before approval, sends to denied exits, sends with changed arguments, and sends after relevant policy changes. For exposure, report the number of secret characters or bits recoverable from each public surface, plus exact whole-value matches. For utility, report benign output changed, schema failures caused by filtering, successful intended calls, and additional decisions. Measure provider latency, guard time, substitution time, filtering time, and output size separately. Human decision time must not be merged with host processing time.

Adversarial tools should return direct echoes, every supported encoding, partial strings, split arrays, numeric codes, boolean predicates, digests, errors, and a later-readable stored value. Include the interactive-input route and unexpected protocol data. An attack counts as successful only when a supplied reconstruction program recovers the claimed value or fact from permitted observations. A hash without a demonstrated recovery is reported as a leaked function, not automatically as full recovery.

Ablations remove exit checking, explicit approval, late reading, connection revision checks, individual encoding forms, and proposed manifest fields one at a time. Pause the host at controlled points between checking, approval, reading, and sending. Change grants, item sources, field lists, exits, and connection revisions at those points. These schedules directly test freshness assumptions.

Run each generated case under each applicable variant with matched inputs. Publish seeds, corpus rules, dependency versions, and the complete implementation and configuration used for the study. Report counts and denominators for each attack class, rather than only an average. Use paired differences and bootstrap intervals clustered by generated case for performance and corruption metrics. For a prespecified set of independent trials with no leaks, an exact one-sided binomial bound may summarize that set; it does not prove safety against untested attacks. Repeated encodings of one secret are not independent security trials.

Keep holdout transforms devised after the form list is fixed. Predefine failures and comparisons, and preserve unexpected cases as minimal reproductions. One valid partial echo refutes complete data-loss prevention, even if every direct echo is removed.

IX. Reproducible synthetic vectors

These fixtures require no live credential. Fix organization `o`, item identity `i1`, alias `github`, field `token`, and exit `(c1,`

`create_issue`). Set item policy version and connection revision to one. Use `ABCD1234` as a synthetic secret. Record every provider read and tool send in a private test trace. All other observations come from the program-facing channels. Start each vector with a fresh host state unless a later step explicitly preserves it. The vectors define expected results. The executable subset and its limits are stated below.

Vector 1: substitution and denial. Use arguments `{"authorization": "Bearer ⟦vault:github/token⟧"}`. Before approval, expect no provider read and no tool send. After approval of `c1/create_issue`, expect exactly that field read and an outgoing authorization string `Bearer ABCD1234`. The program's original tree must still hold the reference. Changing the connection to `c2` must reject before either read or send. Changing the field to `password` must also reject. A reference appearing only as an object key is ignored by scanning and remains literal; it must not cause a secret read.

Vector 2: literal and single-pass substitution. Set the synthetic secret to the exact string `$&`. Substitution must produce `$&`, without copying the matched reference as a replacement-language operator might do. Next set the secret to `⟦vault:other/token⟧`. The outgoing string must contain that literal text, with no read of `other`.

Vector 3: redaction and recovery. Restore `ABCD1234`. An echo of the whole value must become `⟦hidden:github/token⟧`. The JSON result `{"left": "ABCD", "right": "1234"}` remains unchanged under the finite form list. Concatenating its two values recovers the entire synthetic secret. The reconstruction program reads `left`, appends `right`, and compares with the fixture.

Vector 4: short forms and key collisions. Set the secret to `ab`. The result list `["ab", "YWI", "YWI=", "6162"]` must retain its first two entries and replace the final two with hidden markers. Next set the secret to `content` and use a schema that requires a key named `content`. Redacting a failure object with that key changes its name, so schema validation must choose the fixed public error. Finally set the secret to `hidden`. Redacting its raw echo inserts a marker that contains `hidden` again. This is a direct counterexample to unconditional whole-value absence after replacement.

Vector 5: policy change and later reads. Pause an approved call after snapshot checking. Remove its exit in current metadata and increase the item policy version. Leave the connection revision unchanged. The defined baseline still sends, because its release check used the snapshot. The proposed manifest rejects the version change before reading. In a separate run, let an allowed tool store the secret and return a constant receipt. Read that store through a later reference-free call. The baseline returns the value because the later call has no per-call secret list.

For each vector, record the input tree, policy snapshot, decision, read trace, outgoing copy, raw result, filtered result, and public observation. Digest encoding must distinguish types and preserve strings, list order, and stored object-member order. Keep member order explicit when testing redaction collisions. An implementation may choose safer collision or error rules, but it must label that choice as a changed algorithm and recheck the related proofs. These vectors do not prove all-path conformance.

A. *Executable evidence and its scope*

We ran a Python standard-library reference model of the substitution and redaction algorithms above. All values are synthetic and grant no access. The fixed vectors supply the edge cases. The following generator gives the remaining inputs; `redact` denotes exactly the finite redactor defined in this paper.

```
for seed in range(30):
    rng = random.Random(seed)
    value = "value/" + "".join(
        rng.choice("abcdef0123456789") for _ in range(24)
    )
    # Test every distinct form defined by the algorithm.
for byte in range(256):
    value = format(byte, "08b")
    reply = {"bit_" + str(i): bool(byte & (1 << i))
             for i in range(8)}
    seen = redact(reply, {"test/value": value})
    rebuilt = sum(1 << i for i in range(8)
                  if seen["bit_" + str(i)])
    assert rebuilt == byte
```

The original-tree and nested-tree checks exercise substitution directly. Other checks cover literal dollar text, single-pass replacement, whole-form removal, split reconstruction, short forms, schema-key removal, a marker that contains the secret, boolean output, and a later result without a secret context. Seeded values vary known encodings, not adversary strength. These are deterministic conformance checks and counterexamples; a count of passed assertions is not a probability of resisting attacks.

With 30 seeds, the run passes 449 assertions. It generates 30 strings, each with the prefix `value/` and 24 hexadecimal characters. After duplicate forms are removed for each string, the corpus contains 150 whole-form echoes. The measured outcome is whether that exact form remains in the output. This corpus is built from the redactor's own form list, so it tests conformance rather than unseen attacks.

TABLE III
MEASURED SYNTHETIC ECHO RESULTS. A ZERO HERE DOES NOT MEASURE SECRET RECOVERY THROUGH OTHER REPRESENTATIONS.

Filter	Whole forms left / cases
No filter	150 / 150
Raw value only	120 / 150
All defined forms	0 / 150

The exhaustive response test uses all 256 byte values. Each secret is its eight-character binary form. The simulated tool returns eight labeled booleans, one per bit. After the full redactor, the reconstruction program recovers 256 of 256 values and observes 256 distinct transcripts. A fixed `done` receipt gives one transcript for the same inputs. This is a small executable witness for Claim 5, not a comparison of full agent systems. Split strings, short forms, and the marker-content example remain separate successful disclosures.

The model does not invoke a live provider, the product approval store, or its transport. It does not establish that every public error and log path passes through the same filter. It also does not implement the proposed manifest or policy lease. Those claims need trace-level product tests with controlled

pauses, independent output capture, and mutation of each bound field. Report such evidence separately from this small model.

X. Limitations and conclusion

This design serves calls that can be assembled without learning their credential. It cannot support secret-dependent local computation under the same claim. It depends on trusted host and provider code, safe transport, correct policy configuration, and full mediation of release paths. It does not erase secrets from host memory, hide access patterns, or control an approved tool after release. Plaintext retention, debugging, media output, and external stores need their own analysis.

Agent-written code can name and place a secret without receiving its value. The conditional proofs cover secrecy before release and direct destinations. A manifest can strengthen what an approval binds. Finite result redaction remains useful for common echoes, but the worked attacks show why it is not complete data-loss prevention. The decisive trust choice is the tool that receives the value.

References

[1] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.

[2] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical Capabilities for UNIX," in *19th USENIX Security Symposium*, USENIX Association, 2010. [Online]. Available: https://www.cl.cam.ac.uk/research/security/capsicum/papers/2010usenix-security-capsicum-website.pdf

[3] D. E. Denning, "A Lattice Model of Secure Information Flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.

[4] A. Sabelfeld and A. C. Myers, "Language-Based Information-Flow Security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, 2003, [Online]. Available: https://www.cs.cornell.edu/andru/papers/jsac/sm-jsac03.pdf

[5] A. C. Myers and B. Liskov, "Complete, Safe Information Flow with Decentralized Labels," in *Proceedings of the IEEE Symposium on Security and Privacy*, 1998. [Online]. Available: https://www.cs.cornell.edu/andru/papers/sp98/paper.html

[6] A. C. Myers, A. Sabelfeld, and S. Zdancewic, "Enforcing Robust Declassification and Qualified Robustness," *Journal of Computer Security*, vol. 14, no. 2, pp. 157–196, 2006, doi: 10.3233/JCS-2006-14203.

[7] T. Hunt, Z. Zhu, Y. Xu, S. Peter, and E. Witchel, "Ryoan: A Distributed Sandbox for Untrusted Computation on Secret Data," in *12th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2016, pp. 533–549. [Online]. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/hunt

[8] E. Debenedetti *et al.*, "Defeating Prompt Injections by Design." [Online]. Available: https://arxiv.org/abs/2503.18813v2

[9] M. Costa *et al.*, "Securing AI Agents with Information-Flow Control." [Online]. Available: https://arxiv.org/abs/2505.23643v2

[10] HashiCorp, "KV: Secrets Engines: HTTP API." Accessed: Oct. 27, 2026. [Online]. Available: https://developer.hashicorp.com/vault/api-docs/secret/kv/kv-v2

[11] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," technical report RFC 8785, 2020. doi: 10.17487/RFC8785.

[12] RFC Editor, "Errata for RFC 8785." Accessed: Oct. 27, 2026. [Online]. Available: https://www.rfc-editor.org/errata/rfc8785